→ The form mouse is lemma as citation form. mouse is also the lemma for the word mice. Similarly, sing is lemma for sing, sang, sung.

→ Each lemma can have multiple meanings. The lemma mouse can infer rodent as cursor. So lemmas can be polysemous (have multiple senses) can make interpretation difficult.

→ Word Sense disambiguation = The task of determining which sense of a word is being used in a particular context.

→ Synonymy = Relationship b/w word senses.

→ Cat & dogs are synonym but they are similar words. Coffee is not similar to cup (don't share any feature) but they are related words.

→ One common kind of relatedness b/w words is if they belong to the same semantic field. A semantic field is a set of words which cover a particular semantic domain & bear structured relat's with each other.

→ Semantic fields are also related to topic models like Latent Dirichlet Allocation (LDA) which apply unsupervised learning to include associated words from text.

→ Semantic fields are also related to topic models which apply unsupervised learning to include associated words from text.

→ The word connotation has diff. meanings in diff. fields.

→ 3 imp. dimensions of affective meaning ↳ valence = The pleasantness of stimulus
↳ arousal = Intensity of emotion provoked by stimulus. ↳ dominance = degree of control
Excited = high arousal, Control = high dom
happy/satisfied have high valence., ~~Excited~~ Excited = high arousal, Control = high dom
↳ So make every word with 3 dimension vector of above with some number.

→ Vector Semantics is the standard way to represent word meaning in NLP. Idea is, two words that occur in very similar distribut's (whose neighbor words are similar) have similar meanings.

→ Represent a word as a point in a multidimensional semantic space.

Vectors for representing words are called embeddings. (mapping from one space or structure to another)

(good, wonderful) → words with similar meaning are nearby in space

(not good, bad, worst)

→ TF-Idf (baseline) = the meaning of a word is defined by a simple funct^n of counts of nearby words.

→ Vectors or distributional models of meaning are generally based on a co-occurrence matrix, a way of representing how often words co-occur.

→ Term-document Matrix = Each row = word & col^m = document

→ 2 documents that are similar will tend to have similar words & so column vector will tend to similar. Same principle applies to words :- similar words have similar vectors bcoz they tend to occur in similar docum^s

→ Term-Term Matrix (Word-Word Matrix) / Term-Context Matrix = both rows & columns have words. [Dimension = |V|×|V|] & entry is no. of times the row (target) word & column (context) word co-occur in same context.

→ Cosine Similarity. [C
To measure similarity b/
that takes 2 vectors (
of their similarity. most
b/w vectors.
↳ Dot product / Inner product
↳ when it is zero ⇒ strong
↳ more frequent words
more words & have high
dot product thus will be
↳ we modify dot produ

⇒ TF-IDF : Weighing term
↳ Raw freq, is not the best
↳ defined when dimensions
↳ TF-IDF = TF * IDF. → TF
↳ To squash the raw freq.
document, doesn't make that
So, $tf_{t,d} = \log_{10}(\text{Count}(t$
↳ IDF is used to give a hi
Terms that are limited to a f
from rest of collect^n; term
are not as helpful.
↳ document frequency (df_t)
→ $idf_t = \log_{10}\left(\frac{N}{df_t}\right)$. So.
→ tf-idf is a weighted val

⇒ Pointwise Mutual Info^m (
↳ Alternative weighting funct^n
↳ Best way to weight the a
two words co-occur in ou
appear by chance.
↳ Mutual Info^m $I(x,y) = \log$
↳ $PMI(w,c) = \log_2 \frac{P(w,c)}{P(w)}$
↳ $PPMI(w,c) = \max(\log$
→ PMI has the prob^m of betn
to have very high PMI values.

⇒ Cosine Similarity. [$\cos\theta = \frac{a \cdot b}{|a||b|}$, b/w -1 & 1 (same direct^n)] [-1 = opposite direct^n, 0 = orthogonal]

To measure similarity b/w 2 target words v & w, we need a metric that takes 2 vectors (of words or documents dimension) & gives a measure of their similarity. Most common similarity metric is cosine of angle b/w vectors.

↳ Dot product/Inner product $(v, w) = v \cdot w = \sum_{i=1}^{n} v_i w_i$ [Dot product acts as a similarity metric]

↳ when it is zero ⇒ strong dissimilarity.

↳ More frequent words have longer vectors, since they tend to co-occur with more words & have high-co-occurence values with each of them. The raw dot product thus will be higher for frequent words.

↳ we modify dot product to normalize for vector length by dividing by length

⇒ TF-IDF : Weighing terms in the vector.

↳ Raw freq. is not the best measure of association b/w words.

↳ defined when dimensions = documents. When dimensions = words, we use PPMI (algo)

↳ TF-IDF = TF * IDF. → TF = Freq. of word t in document d. $tf_{t,d}$ = count(t, d).

↳ To squash the raw freq., we use $\log_{10}$ bcoz if a word appears 100 times in a document, doesn't make that word 100 times more likely to be relevant in document.

So, $tf_{t,d} = \log_{10}(\text{count}(t,d) + 1)$.

↳ IDF is used to give a higher weight to words that occur only in a few documents. Terms that are limited to a few documents are useful for discriminating those docs from rest of collect^n; terms that occur frequently across the entire collection are not as helpful.

↳ document frequency (df_t) of a term 't' is no. of documents it occurs in.

→ $idf_t = \log_{10}\left(\frac{N}{df_t}\right)$. So. $w_{t,d} = tf_{t,d} * idf_t$.

→ tf-idf is a weighted value $w_{t,d}$ for word t in document d.

⇒ Pointwise Mutual Info^n (PMI)

↳ Alternative weighting funct^n to tf-idf, PPMI (Positive PMI) is used for term-term matrix

↳ Best way to weight the association b/w 2 words is to ask how much more the two words co-occur in our corpus than we would have a priori expected them to appear by chance.

↳ Mutual Info^n $I(X,Y) = \sum_x \sum_y P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}$.

↳ $PMI(w, c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$ [c = context word] [Ratio gives an estimate of how much more the 2 words co-occur than we expect by chance.]

↳ $PPMI(w, c) = \max\left(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0\right)$.

→ PMI has the prob^m of being biased toward infrequent events. Very rare words tend to have very high PMI values. Sol^n $PPMI_\alpha(w,c) = \max\left(\log_2 \frac{P(w,c)}{P(w)P_\alpha(c)}, 0\right)$, where, $P_\alpha(c) = \frac{\text{Count}(c)^\alpha}{\sum \text{count}(c)^\alpha}$

→ Cosine Similarity. $\left\{ \cos\theta = \frac{a \cdot b}{|a||b|} \right.$, b/w -1 & 1 $\left.\right]$ (same direction) $\left(\begin{array}{l} -1 = \text{opposite direct}^n \\ 0 = \text{orthogonal} \end{array} \right)$

To measure similarity b/w 2 target words v & w, we need a metric that takes 2 vectors (of words or documents dimensions) & gives a measure of their similarity. Most common similarity metric is cosine of angle b/w vectors.

↳ Dot product/Inner product $(v, w) = v \cdot w = \sum_{i=1}^{n} v_i w_i$ [Dot product acts as a similarity metric]

↳ when it is zero ⇒ strong dissimilarity.

↳ More frequent words have longer vectors, since they tend to co-occur with more words & have high-co-occurrence values with each of them. The raw dot product thus will be higher for frequent words.

↳ we modify dot product to normalize for vector length by dividing by length

⇒ TF-IDF : Weighing terms in the vector.

↳ Row freq is not the best measure of association b/w words.

↳ defined when dimensions = documents. When dimensions = words, we use PPMI (algo)

↳ TF-IDF = TF * IDF. → TF = Freq. of word t in document d. $tf_{t,d}$ = count (t, d).

↳ To squash the raw freq., we use $\log_{10}$ bcoz if a word appears 100 times in a document, doesn't make that word 100 times more likely to be relevant in document.

So, $tf_{t,d} = \log_{10}(\text{count}(t,d) + 1)$.

↳ IDF is used to give a higher weight to words that occur only in a few documents. Terms that are limited to a few documents are useful for discriminating those docs from rest of collectn; terms that occur frequently across the entire collection are not as helpful.

↳ document frequency (df_t) of a term 't' is no. of documents it occurs in.

→ $idf_t = \log_{10}\left(\frac{N}{df_t}\right)$. So. $w_{t,d} = tf_{t,d} * idf_t$.

→ tf-idf is a weighted value $w_{t,d}$ for word t in document d.

⇒ Pointwise Mutual Infom (PMI)

↳ Alternative weighting functn to tf-idf, PPMI (Positive PMI) is used for term-term matrix

↳ Best way to weight the association b/w 2 words is to ask how much more the two words co-occur in our corpus than we would have a priori expected them to appear by chance.

↳ Mutual Infom $I(X,Y) = \cancel{\log_2 P} \sum_x \sum_y P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}$.

↳ $PMI(w, c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$ [c = context word] [Ratio gives an estimate of how much more the 2 words co-occur than we expect by chance.]

↳ $PPMI(w, c) = \max\left(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0\right)$.

→ PMI has the probm of being biased toward infrequent events. Very rare words tend to have very high PMI values. Soln $PPMI_\alpha(w,c) = \max\left(\log_2 \frac{P(w,c)}{P(w)P_\alpha(c)}, 0\right)$, where, $P_\alpha(c) = \frac{\text{Count}(c)^\alpha}{\sum \text{count}(c)^\alpha}$

→ Word2Vec

↳ It is a more powerful word representatⁿ. embeddings, short dense vectors (dimension 50-1000), rather than much large vocab size |V| or no. of documents D from 50-1000, rather than much large vocab size |V| or no. of documents D

↳ Dense vectors work better in every NLP task than sparse vectors.

↳ Representing words as 300-dimensional dense vectors requires our classifier to learn far fewer weights than if we represented words as 50000 dimensional & so it avoids overfitting.

→ Skip-gram with negative sampling (SGNS) ⇒ Skip-gram algoⁿ is one of two algoⁿ in a SW package called word2vec & sometimes algoⁿ is referred as word2vec.

→ word2vec embeddings are static embeddings [fixed embeddings] for each word in vocab.. Dynamic Contextual embeddings includes BERT in which vector for each word in different in different contexts.

→ Here, we take learned classifier weights as word embeddings.

→ Intuition - we can just use running text as implicitly supervised training data for such a classifier; a word 'c' that occurs near the target word acts as gold 'correct answer' to the question. This method, often called "self-supervision" avoids the need for any sort of Hand-labeled supervision signal.

→ Intuition of skip-gram:

① Treat the target word & a neighboring context word as +ve examples.

② Randomly sample other words in the lexicon to get -ve "

③ Use logistic regression to train a classifier to distinguish these 2 cases

④ Use the learned weights as embeddings.

Ex. ... lemon., a [tablespoon of apricot jam. a] pinch } say window = ±2 for context words

c_1 c_2 w c_3 c_4 (w = target word)

↳ We need to find $P(+|w,c)$ [c is a real context word for w]

& $P(-|w,c) = 1 - P(+|w,c)$ [" " not " " " "]

↳ Similarity $(w,c) \approx c.w$ ⇒ Prob $= \sigma(c.w) = P(+|w,c) = \sigma(c.w) = \frac{1}{1+\exp(-w.c)}$

↳ Skip-gram makes assumption that all context words are independent.

So, $P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i.w)$.

→ Skip-gram actually stores 2 embeddings for each word, one for the word as a target & one for word as context. & it uses more -ve examples than +ve.

→ Loss Functⁿ $L_{CE} = -\log\left[P(+|w, c_{positive}) \prod_{i=1}^{k} P(-|w, c_{negative_i})\right]$

That is, we want to maximize the dot product of the word with actual context words & minimize the dot products of the word with k negative sampled non-neighbor words.

→ Skipgram model learns 2 separate embeddings for each word i: the target embedding w_i & the context embedding c_i, & then add $w_i + c_i$ to represent word w_i.

→ fasttext, also a stat & Glove [Global vectors] f

the intuitions of count-b

linear structures used by

→ Visualize Embeddings..

→ Word2gram model. if a is

↳ Ex. King − man + wo

similarly. Paris − France

→ a : b :: a* : b* ⇒ b*

(*) Neural N/ws & Neura

→ Neural N/ws are a funda

they are called neural bcoz

→ Feedforward N/w = Computa

→ tanh fun has the nice pro

& mapping outlier values t

→ The Rectifier fun, has nice p

→ In tanh & sigmoid, zero d

we train n/w by propaga

(partial derivatives) from each

until it is too small to b

↳ ReLU don't have this

↳ XOR is not linearly sepa

→ In multinomial classific

→ One of the reasons we

each layer in a NN is tho

to a single layer n/w.. So,

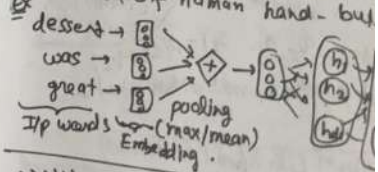
Just a notational variant o

→ We use deep learning's

representing words as embedd

Instead of human hand-bu

Ex



→ NNs can be used for language m

→ NNs are less interpreta

→ Ex. cat feds food & if we

→ Loss funct = Cross Entropy L

$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \times \frac{\partial z}{\partial x}$

→ fastText & also a static embeddings like word2vec & GloVe (Global vectors) → GloVe is based on ratio of probs from word-word co-occurrence matrix, combining the intuitions of count-based models like PPMI while also capturing linear structures used by methods like word2vec.

→ Visualize Embeddings. → Project its dimensions of a word to 2 dimensions using t-SNE.

→ Analogy model. if a is to b then a^* is to what? → apple : tree :: grape : ?

↳ Ex. $\overrightarrow{King} - \overrightarrow{man} + \overrightarrow{woman}$ is a vector close to $\overrightarrow{queen}$.

similarly $\overrightarrow{Paris} - \overrightarrow{France} + \overrightarrow{Italy}$ " " " " $\overrightarrow{Rome}$

apple → tree, grape → vine.

→ $a : b :: a^* : b^* \Rightarrow \hat{b}^* = \underset{x}{\operatorname{argmin}}\ \text{distance}(x, b - a + a^*)$.

Neural N/ws & Neural Language Models.

→ Neural N/ws are a fundamental computational tool for language processing & they are called neural bcoz their origins lie in McCulloch-Pitts neuron.

→ Feedforward N/w = Computat^n proceeds iteratively from one layer to next. deep = many layers.

→ tanh fun has the nice properties of being smoothly differentiable & mapping outlier values toward the mean.

→ The Rectifier fun, has nice properties that result is close to linear.

→ In tanh & sigmoid, zero derivatives for large z cause prob^m for learning bcoz we train n/w by propagating error signal backwards, multiplying gradients (partial derivatives) from each other causes error signal to get smaller & smaller until it is too small to be used for training. [Vanishing Gradient Prob^m]

↳ ReLU don't have this prob^m.

↳ XOR is not linearly separable Funct^n & soln is use 3 ReLU units in 2 layers.

→ In multinomial classificat^n like PoS tag, o/p value is prob. of that PoS & sum = 1

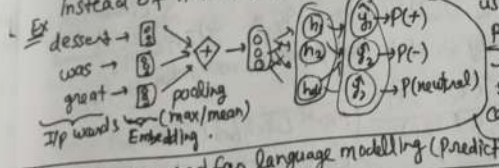
→ One of the reasons we use non-linear activat^n funct^n for each layer in a NN is that if we did not, the resulting n/w is exactly equivalent to a single layer n/w. So, w/o non-linear activat^n fun, a multilayer n/w is just a notational variant of a single layer N/w. bcoz $z^{[1]} = W^{[1]}x + b^{[1]}$ & $z^{[2]} = W^{[2]}z^{[1]} + b^{[2]}$

$z(W) = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} = W'x + b'$

→ We use deep learning's ability to learn features from data by representing words as embeddings like word2vec/GloVe embeddings instead of human hand-built features.

→ Use already learned weights (an embedding representation) using word2vec/GloVE for input words is called pretraining.

→ Using pretraing embedding represent^n whether simple static word embedding can powerful contextual embeddings is one of the central ideas of DL.



→ NNs can be used for language modelling (Predicting words from prior word context.

→ NNs are less interpretable than n-gram models $P(w_i = i \mid w_{t-3}, w_{t-2}, w_{t-1})$ I/P for language model

→ Ex. cat feeds food & if we see dog feeds → here also food predicted bcoz cat & dog have same embedding.

→ Loss funct^n = Cross Entropy Loss, Backprop = Backward different^n (depends on computational graph) Ex $L(a,b,c) = c(a+2b)$

$d = 2b$, $e = a+d$, $L = ce$

$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \times \frac{\partial e}{\partial d} \leftarrow \partial L/\partial e \leftarrow$

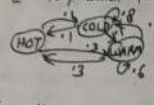
→ Hyperparameters in NN ≡ learning rate, mini-batch size, # layers, units per layer, activation fn

(*) Sequence Labeling for PoS & NER (Named Entity Recogⁿ)

→ PoS are generally assigned to individual words or morphemes.
→ A proper name is often an entire multiword phrase like "Marie Curie", a location or organizatⁿ. Named Entity refered to a proper name.
→ Sequence Labelling Algoⁿ = Hidden Markov Model (HMM) or Conditional Random Field (CRF) (discriminative algo) (generative algo)
→ In PoS tagging, Input is a sequence $x_1, x_2, \ldots, x_n$ of tokenized words & O/p is a sequence $y_1, y_2, \ldots, y_n$ of tags for each word.
→ A sequence labeler (HMM, CRF, RNN, Transformer etc) is trained to label each tag in a sequence (with tag)
→ An HMM is a probabilistic sequence model: given a sequence of units (words, letters, morphemes, sentences etc), it computes a prob. distn over possible sequences of labels & chooses the best label sequence.
→ The HMM is based on augmenting the Markov chain. A Markov chain models tells something about probs of sequences of RVs, states, each of which can take on values from some set. These set can be words or tags or ~~anything~~ symbols representing anything, for ex, weather.

↳ A markov chain has a very strong assumptⁿ if we want to predict future in the sequence, all that matter is current state. All states before the current state have no impact on future except via current state.

Ex. To predict tomorrow's weather, today's weather only matters, not yesterday's



→ Start Distributⁿ $\pi = [0.1, 0.7, 0.2]$ ↳ start with state ②.

→ Assumptⁿ = Past does not matter, only present matters
↳ $P(q_i = a \mid q_i \ldots q_{i-1}) = P(q_i = a \mid q_{i-1})$ { Bigram language model

↳ Formally. A Markov chain is specified by following components:
① $Q = q_1 q_2 \ldots q_N$ (N states) ② $A = a_{11} a_{12} \ldots a_{NN}$ (Transition prob. matrix) ③ $\pi = \pi_1, \pi_2, \ldots$ (initial prob dist) $\sum \pi = 1$

→ The HMM ↳ In many cases, the events we are interested, are hidden (here tags are hidden)

Components ① $Q = q_1 \ldots q_N$ (N states) ② $A = a_{11} \ldots a_{NN}$ = transition prob. matrix.
③ $O = o_1 o_2 \ldots o_T$ (T observatⁿ) ④ $B = b_i(o_t)$ = Emission Probs/observatⁿ likelihoods, each expressing the prob. of an observatⁿ o_t being generated from a state q_i ⑤ $\pi = \pi_1, \pi_2, \ldots$ (initial prob dist) $\sum \pi = 1$

↳ Markov Assumptⁿ ① $P(q_i \mid q_1 \ldots q_{i-1}) = P(q_i \mid q_{i-1})$ ② $P(o_i \mid q_1 \ldots q_T, o_1 \ldots o_i) = P(o_i \mid q_i)$
i.e. Prob. of an o/p observatⁿ o_i depends only on state that produce, not other states.

→ So ① A = transition Prob. Matrix $P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$ (Maxᵐ Likelihood Estimate)
② B = Emission " " $P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$

→ Decoding = Determ/ⁿ hidden variable. Viterbi Algoⁿ is decoding algoⁿ based on DP.

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\arg\max}\, P(t_1 \ldots t_n \mid w_1 \ldots w_n) = \underset{t_1 \ldots t_n}{\arg\max} \frac{P(w_1 \ldots w_n \mid t_1 \ldots t_n)\, P(t_1 \ldots t_n)}{P(w_1 \ldots w_n)} = \underset{t_1 \ldots t_n}{\arg\max}\, P(w_1 \ldots w_n \mid t_1 \ldots t_n)\, P(t_1 \ldots t_n)$$

Now, Assumptⁿ ⇒ $P(w_1 \ldots w_n \mid t_1 \ldots t_n) \approx \prod_{i=1}^{n} P(w_i \mid t_i)$ & $P(t_1 \ldots t_n) \approx \prod_{i=1}^{n} P(t_i \mid t_{i-1})$ { Bigram Assumⁿ

So, $\hat{t}_{1:n} \approx \underset{t_1 \ldots t_n}{\arg\max} \prod_{i=1}^{n} P(w_i \mid t_i) * P(t_i \mid t_{i-1})$ = ~~Transit~~ Emission Prob * Transitⁿ Prob.

(*) RNN & LSTMs

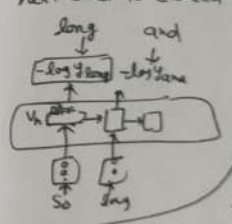
→ We comprehend & produce
→ RNN is any N/w tha
value of some unit is d

→ The h form o the de

→ Adding the temporal non-recurrent architectur performing the standard fe

$h_t =$
$y_t =$

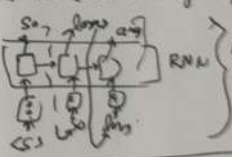
→ We always give the m next word is called teache



in nex gene

→ A at ti times

linear (bcoz many layer of



Auto reg Gene

→

(*) LSTM (Long Short Term Memory) →

↳ Captures distant Infom
↳ Forget infom which is not rele & remember infom required
↳ LSTM removes Infom th from context & add infom th for later decision making.
→ Gates = NN. → Choice of s arises from its tendency to

① Forget Gate $f_t = \sigma(U_f$
$k_t = C_{t-1} \odot$

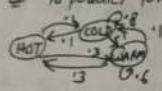
② Add Gate. $g_t = \tanh(U_g h_{t-}$
$I_t = g_t \odot i_t$

③ O/P Gate. $O_t = \sigma(U_o h_{t-1} +$

→ Hyperparameters in NNs (learning rate, mini-batch size, # layers, units per layer, activation ...)

(*) Sequence Labeling for POS & NER (Named Entity Recogⁿ)

→ POS are generally assigned to individual words or morphemes.
→ A proper name is often an entire multiword phrase like "Marie Curie", a location or organizatⁿ. Named Entity refered to a proper name.
→ Sequence Labelling Algoⁿ = Hidden Markov Model (HMM) or Conditional Random Field (CRF) (discriminative algo) (generative algo)
→ In POS tagging, Input is a sequence $x_1, x_2, \dots, x_n$ of tokenized words & O/p is a sequence $y_1, y_2, \dots, y_n$ of tags for each word.
→ A sequence labeler (HMM, CRF, RNN, Transformer etc) is trained to label each in a sequence with the
→ An HMM is a probabilistic sequence model: given a sequence of units (words, letters, morphemes, sentences etc), it computes a prob. distn over possible sequences of labels & chooses the best label sequence.
→ The HMM is based on augmenting the Markov chain. A Markov chain model tells something about probs of sequences of rvs, states, each of which can take on values from some set. These set can be words or tags or ~~anything~~ symbols representing anything, for ex. weather.
↳ A markov chain has a very strong assumptⁿ if we want to predict future in the sequence, all that matter is current state. All states before the current state have no impact on future except via current state.
Ex. To predict tomorrow's weather, today's weather only matters, not yesterday's



→ Start Distributⁿ $\pi = [0.1, 0.7, 0.2]$
↳ start with state ②.
→ Assumptⁿ = Past does not matter, only present matters
↳ $P(q_i = a \mid q_i \dots q_{i-1}) = P(q_i = a \mid q_{i-1})$ { Bigram language model

↳ Formally - A Markov chain is specified by following components:
① $Q = q_1 q_2 \dots q_N$ (N states) ② $A = a_{11} a_{12} \dots a_{NN}$ (Transition prob. matrix) ③ $\pi = \pi_1, \pi_2, \dots$ (Initial prob dist) $\sum \pi = 1$

⇒ The HMM ↳ In many cases, the events we are interested, are hidden (here tags are hidden)
Components ① $Q = q_1 \dots q_N$ (N states) ② $A = a_{11} \dots a_{NN}$ = transition prob. matrix.
③ $O = o_1 o_2 \dots o_T$ (T observatⁿ) ④ $B = b_i(o_t)$ = Emission Probs/observatⁿ likelihoods, each expressing the prob. of an observatⁿ o_t being generated from a state q_i ⑤ $\pi = \pi_1, \pi_2, \dots$ (Initial prob dist) $\sum \pi = 1$
↳ Markov Assumptⁿ ① $P(q_i \mid q_1 \dots q_{i-1}) = P(q_i \mid q_{i-1})$ ② $P(o_i \mid q_1 \dots q_T, o_1 \dots o_T) = P(o_i \mid q_i)$
i.e. Prob. of an o/p observatⁿ o_i depends only on state that produce, not other states.
→ So ① A = transition Prob. Matrix $P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$ (Maxᵐ Likelihood Estimate)
② B = Emission " " $P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$.

→ Decoding = Determinⁿ hidden variable. Viterbi Algoⁿ is decoding algoⁿ based on DP.

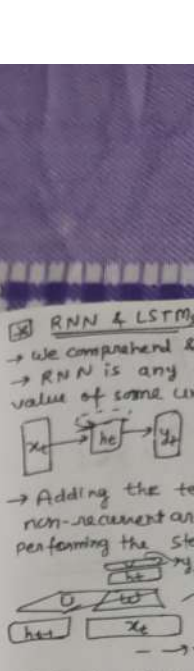
$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\arg\max}\, P(t_1 \dots t_n \mid w_1 \dots w_n) = \underset{t_1 \dots t_n}{\arg\max} \frac{P(w_1 \dots w_n \mid t_1 \dots t_n)\, P(t_1 \dots t_n)}{P(w_1 \dots w_n)} = \underset{t_1 \dots t_n}{\arg\max}\, P(w_1 \dots w_n \mid t_1 \dots t_n)\, P(t_1 \dots t_n)$$

Now, Assumptⁿ ⇒ $P(w_1 \dots w_n \mid t_1 \dots t_n) \approx \prod_{i=1}^{n} P(w_i \mid t_i)$ & $P(t_1 \dots t_n) \approx \prod_{i=1}^{n} P(t_i \mid t_{i-1})$ { Bigram Assumⁿ

So, $\hat{t}_{1:n} \approx \underset{t_1 \dots t_n}{\arg\max} \prod_{i=1}^{n} P(w_i \mid t_i) * P(t_i \mid t_{i-1})$ = Transitⁿ Emission Prob * Transitⁿ Prob.

(*) RNN & LSTMs

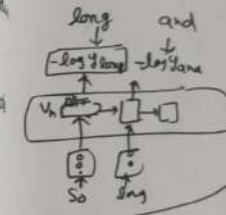
→ We comprehend & produce be
→ RNN is any N/w that c
value of some unit is dire



→ The hidd form of the decist
→ Adding the temporal di non-recurrent architectures. performing the standard feed
$h_t = g($
$y_t = f$
→ We always give the mod next word is called teacher
long and
$-\log y_{long}$ $-\log y_{lane}$
So long

→ T inore next generat
→ Aut at time times t

linear (bcoz many layer of n



Autoregre Genera
→ S

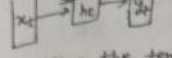
(*) LSTM (Long Short Term Memory)

→ Bi
↳ Captures distant Infoⁿ
↳ Forget infoⁿ which is not needed & remember infoⁿ required fo
↳ LSTM removes infoⁿ that from context & add infoⁿ that for later decision making.
→ Gates = NN. → Choice of si arises from its tendency to p
① Forget Gate $f_t = \sigma(U_f h$
$k_t = C_{t-1} \odot$
② Add Gate. $g_t = \tanh(U_g h_{t-1}$
$I_t = g_t \odot i_t$
③ O/P Gate. $O_t = \sigma(U_o h_{t-1} +$

⊠ RNN & LSTMs [Language = Temporal Phenomenon]

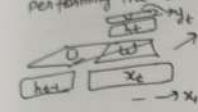
→ We comprehend & produce both spoken & written language as a continuous I/p stream.

→ RNN is any N/w that contains a cycle within its N/w connections means that value of some unit is directly or indirectly dependent on its own earlier O/p as inputs.



→ The hidden layer from the prev. time step provides a form of memory or context that encodes earlier processing & informs the decisions to be made at later points in time.

→ Adding the temporal dimension makes RNNs to be more complex than non-recurrent architectures. But in reality, they are not all that diff. We are still performing the standard feedforward calculation.



$h_t = g(Uh_{t-1} + Wx_t)$

$y_t = f(Vh_t)$

& so on.

→ In language models for RNNs.

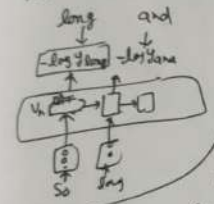
$e_t = Ex_t$

$h_t = g(Uh_{t-1} + We_t)$

$y_t = \text{softmax}(Vh_t)$

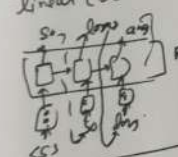
→ We always give the model the correct history of sequence to predict the next word is called teacher forcing.

Replace V with E^T is called weight tying.



→ Today, the approach of using a language model to incrementally generate words by repeatedly sampling the next word conditioned on prev. choice is called autoregressive generation or causal LM generation.

→ Autoregressive model is a model that predicts a value at time 't' based on a linear funct^n of prev. values at times t-1, t-2, & so on. Although language models are not linear (bcoz many layer of non-linearities), we refer it as autoregressive generation.



Autoregressive Generat^n → Useful in machine translat^n, summarizat^n, quest^n answering etc.

→ Stacked RNN = O/p from RNN as input sequence to another RNN.

↳ generally outperform single layer N/w. } optional no. of stacked RNN depends on app^n & training set.

→ Bidirectional RNN. = both left-to-right & right-to-left RNNs & concatenate it.

⊠ LSTM (Long Short Term Memory)

↳ Captures distant info^n

↳ Forget info^n which is not needed for decisions still to come & remember info^n required for decisions still to come.

↳ LSTM removes info^n that is no longer needed from context & add info^n that is likely to be needed for later decision making.

→ Gates = NN. → Choice of sigmoid activat^n funct^n arises from its tendency to push o/p to either 0 or 1.

① Forget Gate $f_t = \sigma(U_f h_{t-1} + W_f x_t)$

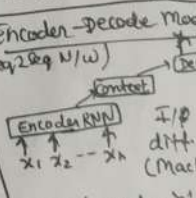
$K_t = C_{t-1} \odot f_t$ (for masking)

② Add Gate. $g_t = \tanh(U_g h_{t-1} + W_g x_t)$ & $i_t = \sigma(U_i h_{t-1} + W_i x_t)$

$J_t = g_t \odot i_t \Rightarrow C_t = J_t + K_t$

③ O/p Gate. $O_t = \sigma(U_o h_{t-1} + W_o x_t)$ & $h_t = O_t \odot \tanh(C_t)$

(⊠) Encoder-Decoder Model (Seq2Seq N/w)



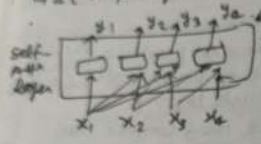
I/p & O/p have diff. length. (Machine Translat^n)

Context vector 'c' is a funct^n of h_i^n, conveys essence of I/p. to decoder. Add to each layer of -

Transformer (~~the II-th feat~~)

↳ It maps sequences of I/P vectors to seq. of o/p vectors of same length

(Causal or masked) self-Attention model.



← Processing happens in $\|^{el}$.

→ Comparison ⇒ $\text{score}(x_i, x_J) = x_i \cdot x_J$.

→ $\alpha_{iJ} = \text{softmax}(\text{score}(x_i, x_J))$ $\forall J \leq i$

→ $y_i = \sum_{J \leq i} \alpha_{iJ} x_J$.

⇒ $\text{Self-Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$.